

AIM: Write a program to implement a linear regression model on the given dataset and find R2 score.

CODE:

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

temperature_df = pd.read_csv("daily-minimum-temperatures-in-me.csv")

temperature_df["Date"] = pd.to_datetime(temperature_df["Date"], errors="coerce")
temperature_df["Daily minimum temperatures"] = pd.to_numeric(
    temperature_df["Daily minimum temperatures"], errors="coerce"
)

temperature_df = temperature_df.dropna()

temperature_df["day_index"] = (
    temperature_df["Date"] - temperature_df["Date"].min()
).dt.days

feature_X = temperature_df[["day_index"]]
target_y = temperature_df["Daily minimum temperatures"]

X_train_split, X_test_split, y_train_split, y_test_split = train_test_split(
    feature_X, target_y, test_size=0.3, random_state=42
)

lr_model_split = LinearRegression()
lr_model_split.fit(X_train_split, y_train_split)

y_pred_split = lr_model_split.predict(X_test_split)
r2_score_split = r2_score(y_test_split, y_pred_split)

print(f"R2 Score (Train-Test Split): {r2_score_split}")
lr_model_full = LinearRegression()
lr_model_full.fit(feature_X, target_y)
y_pred_full = lr_model_full.predict(feature_X)
r2_score_full = r2_score(target_y, y_pred_full)

print(f"R2 Score (Full Data): {r2_score_full}")
```

OUTPUT:

```
R2 Score (Train-Test Split): 6.481178477035776e-05
R2 Score (Full Data): 0.00010567974538144842
```

AIM: Write a program to implement a linear regression model on the given dataset and find the R2 score. (HARDCODE).

CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

class ManualLinearRegression:
    def __init__(self):
        self.slope_m = None
        self.intercept_b = None

    def fit(self, X_train_input, y_train_input):
        X_array = np.array(X_train_input).flatten()
        y_array = np.array(y_train_input)

        x_mean_val = np.mean(X_array)
        y_mean_val = np.mean(y_array)

        numerator_val = np.sum((X_array - x_mean_val) * (y_array - y_mean_val))
        denominator_val = np.sum((X_array - x_mean_val) ** 2)

        self.slope_m = numerator_val / denominator_val
        self.intercept_b = y_mean_val - self.slope_m * x_mean_val

    def predict(self, X_test_input):
        X_array = np.array(X_test_input).flatten()
        return self.slope_m * X_array + self.intercept_b

    def r2_score(self, y_true_input, y_pred_input):
        y_true_array = np.array(y_true_input)
        ss_total_val = np.sum((y_true_array - np.mean(y_true_array)) ** 2)
        ss_residual_val = np.sum((y_true_array - y_pred_input) ** 2)
        return 1 - (ss_residual_val / ss_total_val)

temperature_df = pd.read_csv("daily-minimum-temperatures-in-me.csv")
```

```
temperature_df["Date"] = pd.to_datetime(temperature_df["Date"], errors="coerce")
```

```
temperature_df["Daily minimum temperatures"] = pd.to_numeric(
    temperature_df["Daily minimum temperatures"], errors="coerce"
)
```

```
temperature_df = temperature_df.dropna()
```

```
temperature_df["day_index"] = (
    temperature_df["Date"] - temperature_df["Date"].min()
).dt.days
```

```
feature_X = temperature_df[["day_index"]]
target_y = temperature_df["Daily minimum temperatures"]
```

```
X_train_split, X_test_split, y_train_split, y_test_split = train_test_split(
    feature_X, target_y, test_size=0.3, random_state=42
)
```

```
manual_lr_model = ManualLinearRegression()
manual_lr_model.fit(X_train_split, y_train_split)
```

```
y_predicted_split = manual_lr_model.predict(X_test_split)
r2_value = manual_lr_model.r2_score(y_test_split, y_predicted_split)
```

```
print(f"Slope (m): {manual_lr_model.slope_m}")
print(f"Intercept (b): {manual_lr_model.intercept_b}")
print(f"R2 Score: {r2_value}")
```

OUTPUT:

```
File: C:\Users\ANIRU\Downloads\1111.py
Slope (m): 3.436974780922816e-05
Intercept (b): 11.134647281423998
R2 Score: 6.48117847705798e-05
```

AIM: Write a program to implement a Multiple linear regression model on the given dataset and find the R2 score.

CODE:

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

temperature_df = pd.read_csv("daily-minimum-temperatures-in-me - Copy.csv")

temperature_df.columns = temperature_df.columns.str.strip()
temperature_df = temperature_df.drop(columns=["Unnamed: 2", "Unnamed: 3"], errors="ignore")

temperature_df["Date"] = pd.to_datetime(temperature_df["Date"], errors="coerce")
temperature_df["Daily minimum temperatures"] = pd.to_numeric(
    temperature_df["Daily minimum temperatures"], errors="coerce"
)
temperature_df["Humidity"] = pd.to_numeric(
    temperature_df["Humidity"], errors="coerce"
)

temperature_df = temperature_df.dropna()

temperature_df["day_index"] = (
    temperature_df["Date"] - temperature_df["Date"].min()
).dt.days

feature_X = temperature_df[["day_index", "Humidity"]]
target_y = temperature_df["Daily minimum temperatures"]

X_train_split, X_test_split, y_train_split, y_test_split = train_test_split(
    feature_X, target_y, test_size=0.3, random_state=42
)

lr_model_split = LinearRegression()
lr_model_split.fit(X_train_split, y_train_split)

y_pred_split = lr_model_split.predict(X_test_split)
r2_score_split = r2_score(y_test_split, y_pred_split)

print("R2 Score (Train-Test Split):", r2_score_split)

lr_model_full = LinearRegression()
```

```
lr_model_full.fit(feature_X, target_y)

y_pred_full = lr_model_full.predict(feature_X)
r2_score_full = r2_score(target_y, y_pred_full)

print("R2 Score (Full Data):", r2_score_full)
```

OUTPUT:

```
R2 Score (Train-Test Split): 1.0
R2 Score (Full Data): 1.0
```